Sprint 2 Contribution Log

SOEN 341: CareerConnect
Lamees Ibeid, Backend Team

Overview: In Sprint 2, I worked on the general backend (FastAPI). I built the profile endpoints and the logged-in user helper, the resume endpoints and the apply-for-a-job endpoint, plus a follow-up that saves the optional cover note on applications. I wrote 30 automated tests for this work, and all 53 backend tests pass on my branch. I was also the scribe for the October 16 team meeting.

Total time spent: About 7.5 hours (1 hour on October 16, 3.5 hours on October 22 and 3 hours on October 26).

Meeting Minutes (October 16)

Goal: Record the October 16 team meeting as the scribe.

What I did:

- Took notes during the meeting and wrote up the minutes: attendance, each member's progress, and the decisions (the backend team notifies Sayf about new features so he can write matching tests, and everyone updates their branch with main before opening a pull request).
- Posted them as an issue labelled meeting and added it to the Meeting minutes project, in the same format as the team's earlier minutes.

Time spent: About 1 hour (the meeting and writing up the minutes).

Links:

- Issue "Meeting (16 Oct 26)" in the meeting issues

LA-1: Profile Endpoints and Logged-In User Helper

Goal: A logged-in user can save their profile and read it back.

What I did:

- Logged-in user helper: get_current_user reads the login token from the Authorization header, checks it and returns the user, or 401 "Not logged in." It is shared by every endpoint that needs a logged-in user (App/core/deps.py).

- Profile schemas: ProfileIn with the rules from the API cheat sheet (full name required, summary up to 2000 characters, up to 30 skills) and ProfileOut, which returns empty fields as "" and [] for the frontend (App/schemas/profile.py).
- Profile endpoints: GET /api/v1/profile/me returns the profile or 404 "Profile not created yet."; PUT creates the profile the first time and updates it after that. The user always comes from the token, so users can only reach their own profile (App/routers/profile.py, registered in App/main.py).
- Tests: 9 tests covering the 404 before saving, saving and reading back, updating, the validation rules, privacy between users and requests without a valid token (tests/test_profile.py).

Verification: pytest: 26 passed. Tested by hand in /docs: registered, logged in, authorized with the token, then GET gave 404, PUT gave 200 and GET returned the saved profile.

Time spent: About 3.5 hours (October 22), including setting up my environment again for Sprint 2: pulling main, installing the new packages, updating .env and rebuilding my local database with Alembic and the sample jobs.

Links:

- Commit ba3654d: Add profile endpoints and logged-in user helper
- Pull request #101: Profile endpoints, merged October 26

LA-2: Resume Endpoints

Goal: A logged-in user can upload, list, download, replace and delete their own resumes.

What I did:

- Resume endpoints: POST /api/v1/resumes (upload), GET (list), GET /{id}/download, PUT /{id} (replace) and DELETE /{id} (204), built on Kaila's resume storage functions (App/routers/resumes.py).
- File checks: PDF or Word only, the extension must match, not empty and at most 5 MB, otherwise 422 with a clear message. The limits come from Kaila's rules file (App/resume_rules.py).
- Privacy: a resume that is missing or belongs to someone else returns 404, so resume ids cannot be discovered.
- ResumeOut schema: returns the resume details without the file itself (App/schemas/resume.py).
- Added python-multipart to requirements.txt, which FastAPI needs to receive file uploads (also needed by CI).

- Tests: 11 tests (tests/test_resumes.py).

Verification: pytest: 37 passed. The response fields were checked against the resume type in Krish's frontend code.

Time spent: About 1 hour (October 26).

Links:

- Commit 6d2a223: Add resume endpoints
- Pull request #104: Resume and apply endpoints, merged October 26

LA-3: Apply-for-a-Job Endpoint

Goal: A job seeker can apply to a job with one of their resumes and get a confirmation.

What I did:

- Apply endpoint: POST /api/v1/applications returns 201 with status "Applied" and the message "Application submitted" (App/routers/applications.py).
- Rules: only job seekers can apply (403), the job must exist (404) and be open (422), the resume must belong to the user (404), and applying twice returns 409, including when two requests arrive at the same moment.
- Status constants: the five official statuses in one file, so Youssef's status endpoints can reuse them (App/application_statuses.py).
- Schemas for the request and the confirmation (App/schemas/application.py).
- Tests: 7 tests (tests/test_applications.py). Also updated three profile tests to accept 400 or 422, since the team's error code rule was changing.
- Team coordination: sent Sayf the behaviour of all my endpoints for his acceptance tests, told Youssef about our two versions of the logged-in user helper, and told Krish how to connect his pages.

Verification: pytest: 44 passed. Tested on a running server: upload a resume, apply, then apply again (409).

Time spent: About 0.75 hours, plus about 0.25 hours of team coordination (October 26).

Links:

- Commit 760926b: Add apply-for-a-job endpoint
- Pull request #104: Resume and apply endpoints, merged October 26

Saving the Optional Cover Note

Goal: Accept the optional cover note from the apply dialog (up to 1000 characters) and save it, as requested by Sayf after his tests.

What I did:

- Found that the applications table had no cover_note column, so I added it to the model (App/models/application.py) with a new migration (alembic/versions/b7d4e2a91c05_add_cover_note_to_applications.py) and told Sayf before creating it.
- Accepted the optional field in the schema and saved it in the apply endpoint.
- Tests: 3 tests (saved, optional, over 1000 characters rejected).

Verification: alembic heads showed the new migration, alembic upgrade head applied it to my local database, and pytest: 53 passed. Reminded Sayf that the migration must also be run on the Neon database.

Time spent: About 0.5 hours (October 26).

Links:

- Commit cd8a0dc: Save optional cover note when applying
- Pull request #108: Save optional cover note on applications, merged October 26

Sprint 2 AI Usage Log

Goal: Document my AI use for Sprint 2 in the same format as Sprint 1.

What I did:

- Wrote my Sprint 2 AI usage log covering the profile, resume, apply and cover note work, and uploaded it to AI_Doc.

Time spent: About 0.5 hours (October 26).

Links:

- Commit e78511c: Add Lamees Sprint 2 AI log
- Pull request from branch docs/lamees-sprint2-ai-log

Work Log

October 16, 2026 (1 hour): Attended the weekly team meeting as the scribe, and wrote up and posted the meeting minutes.

October 22, 2026 (3.5 hours): Set up my environment again for Sprint 2 (pulled main, installed the new packages, updated .env, rebuilt my local database with Alembic and loaded the sample jobs). Built the logged-in user helper, the profile schemas and endpoints, and 9 tests. Tested in /docs, committed and opened the pull request.

October 26, 2026 (morning) (2 hours): Built the resume endpoints (11 tests) and the apply endpoint (7 tests) on a branch built on top of the profile work, opened the pull request, and sent the team details about my endpoints.

October 26, 2026 (evening) (1 hour): Added the cover note follow-up requested by Sayf (column, migration, schema, endpoint and 3 tests), fixed the errors I hit along the way, opened the pull request, and uploaded my Sprint 2 AI usage log.